ROUGH DRAFT: one-page brief (for Aarvin to rewrite in his own words)

Scaffold only. Facts are from the repo (decisions.md, errors-found.md, core-idea-and-data-model.md). The ✎ lines are prompts for your own voice; delete them as you go. Target: one page, about 600 words.

Pick an opening (or write your own)

- **A.** "Tax software is rarely wrong in a way that looks wrong. It is wrong at 99% confidence."
- **B.** "A workpaper exists for one reason: to prove the return. Most of them prove that someone ticked a box."
- **C.** "I started by hunting the mistakes, not by drawing screens. The mistakes told me what to build."

✎ *Which moment in this project changed how you saw the problem? Start there.*

1. Approach

- **Mistakes first, screens second.** Before any code, I hunted the packet for confident mistakes: printed pages vs extracted values, the three computed versions, IRS and FTB account records, the prior year, the client's own answers. Every finding went into a ledger with its document, field, value, why it is wrong and the evidence.
- **The findings shaped the model.** Every one of them was a number that looked fine and was not. So every number had to carry its source, its confidence, how it was checked, and the checks it fails.
- **Discovery, then a mock-up, then a thin deployed slice, then rounds.** Ten mock-up screens became three staff screens once I walked the live app myself and asked of each screen "what one question does this answer?"

✎ *What did you cut, and why was cutting it the right call?*

2. Core idea

Every number is a claim that carries its receipt. One click answers four questions: where it came from (page and field), what was done to it (rule, and any change with who, why and the tax effect), who checked it (a sign-off bound to the exact value signed), and what is still open.

Confidence is not proof. The reading software grades its own work; Taxmark checks each number against a second, independent source.

✎ *Say this in the words you'd use to a 20-year CPA over coffee.*

3. Data model (three inputs, one log, views computed per person)

- **Packet:** read-only, never edited. Documents, values, three computed versions, agency records, firm standards.
- **Event log:** append-only. Adjustments, notes, sign-offs, questions, decisions, undo. Nothing is edited or deleted; an undo is a new event on top. **The audit trail is the data, not a feature.**
- **Projection:** for one role, at one point on the story clock: findings, blockers, sign-offs (fresh or stale), adjustments carried through the totals, masking.

- **Claim:** anything that can hold evidence and actions: a line, a document, a finding.
- **Story clock:** one per workspace; the past is view-only, and the return locks when the agency accepts it.

✎ *Draw it by hand? A sketch of "packet + log → view" might say more than the bullets.*

4. Key decisions (and what each cost)

Decision	Trade-off
Checks are deterministic and written against the packet's **schema**, never document ids; each is tested on altered copies of the packet	More test fixtures; in return, they run on the next client
The ledger (my answer key) labels findings but **never creates them**	The app has to re-derive every mistake, honestly
AI reads each document once, on arrival; code verifies every quote before showing it; it never blocks filing	It still misses about half; reported as 13 of 27, not the flattering 17
A change flows through plain sums; a line needing tax computation waits for an engine re-run and blocks approval	No fake recomputation
A role switcher instead of sign-in	One link shows every view; not production auth

5. How I worked with AI (from the meta decision log, broad strokes)

- **Order of authority:** brief → packet and its README → my ledger → research. Anything lower never overrides higher.
- **A different model reviews everything.** Claude Opus 5.5 built; Claude Fable 5.1 reviewed, and had to *probe* the live app, not just read code.
- **Every decision logged the moment it was made**, including the process ones, and including the corrections: the AI quietly moved notifications out of scope, I caught it and it was restored; tests written first but not run red were logged as a slip.
- **The deadline was a backstop, not the driver.** Quality and fit to the brief decided scope.
- **Judge it by using it.** I walked the live app myself before trusting any checklist.

✎ *Where did you overrule the AI, and where did it change your mind?*

6. The errors, and how the design surfaced them

- **27 distinct mistakes** (29 ledger ids; two were the same mistake logged twice). All 27 are flagged on Checks.
- **Each became a deterministic check**, not a memorised answer.
- **The signature example:** California estimated payments, 14,000 on the return, 10,500 on the FTB's own record. The September payment is marked "tax year 2024"; it was read correctly at 0.98 to 0.99. The engine counted payments by date and ignored the year. Confidence gave no warning; the second source did.
- **How it surfaces:** one row per line on Checks, pills for severity, AI, the engine's own warning and **Confident mistake**; one click to the source page; a fix that carries through to the refund; a partner sign-off bound to the new value.

✎ *Pick one more mistake you personally found most satisfying, and say why in two lines.*

Closing line (pick or write your own)

- "A new hire can follow it on day one; a partner can trust what she signs."
- "The packet never changes. Everything people do is on top of it, and nothing is ever lost."